Lecture 26.10: Logistic Regression Scratch Implementation

📌 Topic Overview

In this lecture, Shraddha ma'am writes the complete vectorized Python class for **Logistic Regression from Scratch**, combining sigmoid probabilities, Binary Cross-Entropy gradients, and gradient descent updates.

🏛️ Log-Loss and Gradients

The Binary Cross-Entropy loss is:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Remarkably, taking the derivative with respect to w yields an equation **identical in form to Linear Regression**, but where $\hat{y}$ is passed through the sigmoid function:

$$\frac{\partial J}{\partial w} = \frac{1}{m} X^T (\sigma(Xw + b) - y)$$

💻 Python Implementation

```
import numpy as np

class LogisticRegressionScratch:
    """
    Complete Scratch Implementation of Logistic Regression using Vectorized NumPy:
    - Hypothesis: y_hat = sigmoid(X * w + b)
    - Loss: Binary Cross-Entropy (Log-Loss)
    - Gradients: dw = (1/m) * X^T * (y_hat - y), db = (1/m) * sum(y_hat - y)
    - Updates: w = w - lr * dw, b = b - lr * db
    """
    def __init__(self, learning_rate=0.01, epochs=1000):
```

```
        self.lr = learning_rate
        self.epochs = epochs
        self.w = None
        self.b = None

    def _sigmoid(self, z):
        z_safe = np.clip(z, -500, 500)
        return 1.0 / (1.0 + np.exp(-z_safe))

    def fit(self, X, y):
        m, d = X.shape
        self.w = np.zeros(d)
        self.b = 0.0

        for _ in range(self.epochs):
            # 1. Forward pass: compute probabilities
            linear_model = np.dot(X, self.w) + self.b
            y_hat = self._sigmoid(linear_model)

            # 2. Gradients
            error = y_hat - y
            dw = (1.0 / m) * np.dot(X.T, error)
            db = (1.0 / m) * np.sum(error)

            # 3. Parameter updates
            self.w -= self.lr * dw
            self.b -= self.lr * db

        return self

    def predict_proba(self, X):
        return self._sigmoid(np.dot(X, self.w) + self.b)

    def predict(self, X, threshold=0.5):
        probs = self.predict_proba(X)
        return (probs >= threshold).astype(int)

# --- Function Call & Testing ---
from sklearn.datasets import make_classification
X_data, y_data = make_classification(n_samples=200, n_features=2, n_redundant=0, r

clf = LogisticRegressionScratch(learning_rate=0.1, epochs=1000)
clf.fit(X_data, y_data)
predictions = clf.predict(X_data)
acc = np.mean(predictions == y_data)

print(f"Scratch Logistic Regression Accuracy: {acc*100:.2f}%")
print(f"Learned Weights: {clf.w.round(4)} | Learned Bias: {clf.b:.4f}")
```

🔑 Key Takeaways

1. Gradient updates in Logistic Regression have the exact same matrix form as Linear Regression, with $\hat{y} = \sigma(z)$.
2. Vectorized NumPy avoids nested loops and delivers blazing speed.